AI LAB VIVA

Easy, Humanized One-Line Questions & Answers | Assignments 1–8

Use these short answers for oral revision. Technical terms and key formulas are kept where needed.

Assignment 1 - Simple Reflex Agent

Q1. What is an Intelligent Agent?
Ans: An intelligent agent observes its surroundings using sensors and takes action to achieve a goal.

Q2. What is a Simple Reflex Agent?
Ans: It chooses what to do based only on what it senses right now and on fixed rules.

Q3. What are the main components of an agent?
Ans: The main parts are sensors, rules or a decision-making part, and actuators.

Q4. What is the Vacuum Cleaner World problem?
Ans: It is an AI problem where a vacuum agent moves between rooms and cleans dirty rooms.

Q5. What are the possible actions of the vacuum cleaner agent?
Ans: It can suck up dirt, move left, move right, or stop (NoOp).

Q6. What are the percepts received by the agent?
Ans: It senses which room it is in (A or B) and whether that room is clean or dirty.

Q7. What is meant by the environment in AI?
Ans: The environment is the outside world in which the agent observes things and takes action.

Q8. Why is the Vacuum Cleaner World considered a simple AI problem?
Ans: It has only a few rooms, conditions, and actions, so simple rules can solve it.

Q9. If Room A is Dirty, Room B is Clean, and the vacuum starts at A, what sequence of actions will it perform?
Ans: It cleans Room A, moves right to Room B, and stops when both rooms are clean.

Q10. If both rooms are Dirty and the vacuum starts at B, what will be the sequence of actions?
Ans: It cleans Room B, moves left to Room A, cleans it, and then stops when both are clean.

Q11. If both rooms are already Clean, what should the agent do?
Ans: It should stop by performing NoOp.

Q12. What happens if new dirt appears after the rooms have been cleaned?
Ans: If it senses dirt in its current room, it cleans that room again.

Assignment 2 - Model-Based Intelligent Agent

Q1. What is a model-based intelligent agent?
Ans: It keeps an internal record of the environment and uses that information to decide what to do.

Q2. How does a model-based agent differ from a simple reflex agent?
Ans: A model-based agent remembers past information, while a simple reflex agent uses only what it senses now.

Q3. Why does the agent require an internal state representation?
Ans: It remembers places it has visited and obstacles it has found, even when it cannot see the whole environment.

Q4. How is the grid environment represented in your implementation?
Ans: It is shown as a grid with positions, open cells, obstacles, a starting point, and a goal.

Q5. How does the agent detect and avoid obstacles?
Ans: Before moving, it checks the next cell using its sensors and stored grid information.

Q6. How does the agent update its internal state after each move?
Ans: It records its new position and updates the cells visited and information it knows.

Q7. What are the advantages of using a model-based intelligent agent for grid navigation?
Ans: It remembers obstacles and visited cells, avoids repeating work, and helps it choose better paths.

Assignment 3 - A* Search for the 8-Puzzle

Q1. What are informed and uninformed search techniques?

Ans: Informed search uses extra guidance called a heuristic; uninformed search does not use a heuristic.

Q2. List differences between informed and uninformed search techniques?

Ans: Informed search uses a heuristic to choose promising states. Uninformed search uses the problem rules and path cost, if any.

Q3. What is the evaluation function of A* algorithm?

Ans: A* uses f(n) = g(n) + h(n). Here, g(n) is the cost so far and h(n) estimates the cost left to reach the goal.

Q4. What are the applications of A* algorithm?

Ans: A* is used for finding paths, robot navigation, route planning, games, and puzzles.

Q5. What is problem solving agent?

Ans: It sets a goal and searches for a sequence of actions to reach that goal from the starting state.

Q6. Is A* algorithm informed or uninformed search technique?

Ans: A* is an informed search algorithm because it uses a heuristic.

Q7. What is Best First Search algorithm?

Ans: Best-First Search chooses the next state that looks most promising according to its evaluation rule.

Q8. Where are searching techniques applied in AI?

Ans: They are used in pathfinding, planning, robotics, games, scheduling, and puzzles.

Q9. What are Blind search algorithms?

Ans: Blind search algorithms do not use heuristics. Examples are BFS, DFS, depth-limited search, and iterative deepening.

Q10. What is meant by Heuristic?

Ans: A heuristic is an estimate of how far or how costly it is to reach the goal.

Assignment 4 - Tower of Hanoi by State Space Search

Q1. What is State Space Search?

Ans: It checks possible states and valid moves to find a way from the start to the goal.

Q2. Why is the Tower of Hanoi considered a state-space search problem?

Ans: Each arrangement of disks is a state, and each legal move takes the puzzle to another state.

Q3. What are the components of a state space (state, initial state, goal state, operators)?

Ans: They are the current arrangement, starting arrangement, target arrangement, and legal moves.

Q4. Why is Breadth-First Search (BFS) used in this implementation?

Ans: BFS checks states level by level and finds the shortest path when every move has the same cost.

Q5. Can the Tower of Hanoi be solved using search techniques other than BFS? Name a few.

Ans: Yes. It can also be solved using DFS, iterative deepening, or recursion.

Q6. What is the advantage of BFS over DFS for this problem?

Ans: BFS finds the shortest path when moves have equal cost. DFS might find a longer path first.

Q7. What is the goal state in the Tower of Hanoi problem?

Ans: The goal is to move all disks to the destination peg, with the largest disk at the bottom and smaller disks above it.

Q8. Does BFS always guarantee the shortest sequence of moves? Why?

Ans: Yes, when every move has equal cost, BFS reaches the goal with the fewest moves first.

Q9. What is the time and space complexity of BFS?

Ans: For a graph, BFS takes O(V + E) time and O(V) space, where V is the number of vertices and E is the number of edges.

Q10. What are the applications of State Space Search in Artificial Intelligence?

Ans: It is used in planning, robotics, route finding, puzzles, automated systems, and games.

Q11. Can Tower of Hanoi be solved using recursion? Explain.

Ans: Yes. Move the smaller disks to the spare peg, move the largest disk, then move the smaller disks onto the destination peg.

Q12. Why is Tower of Hanoi considered a classic AI problem?

Ans: It shows how to represent states, follow rules, plan moves, and search for a goal.

Q13. How many minimum moves are required to solve the Tower of Hanoi problem for N disks?

Ans: The minimum number of moves is 2^N - 1.

Assignment 5 - BFS Planning / Blocks World

Q1. What is Breadth-First Search (BFS)?

Ans: BFS explores states level by level using a queue that follows first in, first out (FIFO).

Q2. Why is BFS suitable for solving planning problems such as the 8-puzzle?

Ans: If every move has equal cost, BFS finds the solution with the fewest moves, but it can use a lot of memory.

Q3. What is the difference between a state, initial state, goal state, and action in a planning problem?

Ans: A state is the current arrangement, the initial state is the start, the goal state is the target, and an action changes the arrangement.

Q4. Which data structure is commonly used to implement BFS? Why?

Ans: A FIFO queue is used because it processes states in the order they are added.

Q5. How does BFS determine the shortest solution path?

Ans: It checks all states at one level before moving to the next, so the first goal found has the fewest moves when costs are equal.

Q6. What is the role of the visited/explored list in BFS?

Ans: It keeps track of states already checked so the algorithm does not keep checking them again.

Q7. What will happen if the BFS algorithm does not maintain a visited list?

Ans: It may repeat states, waste time and memory, or get stuck in a loop.

Q8. What are the main advantages and limitations of BFS for solving planning problems?

Ans: BFS finds a shortest path when moves have equal cost, but it can take a lot of time and memory.

Q9. For your selected problem, what is the initial state and what is the goal state?

Ans: At the start, A, B, and C are all on the table. The goal is A on B, B on C, and C on the table.

Q10. What are the possible actions/moves from a given state?

Ans: A clear block can be moved onto the table or onto another clear block, following the rules.

Q11. How is the solution path reconstructed after BFS reaches the goal?

Ans: The algorithm traces back from the goal to the start using parent links, then reverses the moves.

Assignment 6 - Backtracking for 8-Queens

Q1. What is a CSP?

Ans: A Constraint Satisfaction Problem has variables, possible values for them, and rules those values must follow.

Q2. Why is the 8-Queens problem a CSP?

Ans: We place 8 queens on an 8×8 chessboard so that no two queens attack each other.

Q3. What is backtracking?

Ans: Backtracking builds a solution step by step and goes back when a choice breaks a rule or does not work.

Q4. Difference between brute force and backtracking?

Ans: Brute force tries all possible arrangements, while backtracking stops exploring a choice as soon as it becomes invalid.

Q5. What constraints are checked?

Ans: No two queens can be in the same row, column, or diagonal.

Q6. Why only left-side checking?

Ans: Queens are placed from left to right, so only queens already placed on the left need to be checked.

Q7. Time complexity?

Ans: The typical worst-case upper bound for this column-by-column backtracking method is O(N!).

Q8. Applications?

Ans: Backtracking and CSPs are used in Sudoku, scheduling, map colouring, timetables, and resource allocation.

Q9. Number of solutions for 8-Queens?

Ans: There are 92 solutions in total, or 12 unique solutions if rotations and reflections count as the same.

Q10. Difference between Backtracking and Branch & Bound?

Ans: Backtracking rejects choices that break rules. Branch and Bound rejects choices that cannot give a better result.

Assignment 7 - Rule-Based Chatbot

Q1. What is a chatbot?

Ans: A chatbot is a program that talks with users through text or voice.

Q2. What is a rule-based chatbot?

Ans: It gives replies by following fixed rules and matching patterns in what the user types.

Q3. What is pattern matching?

Ans: Pattern matching checks the user's message against stored patterns to choose a reply.

Q4. How does a rule-based chatbot work?

Ans: It receives a message, prepares the text, looks for a matching rule, gives a reply or fallback message, and continues until the user exits.

Q5. What are predefined rules?

Ans: They are fixed if-this-happens, then-give-this-response rules written by the programmer.

Q6. What are the advantages of rule-based chatbots?

Ans: They are simple, fast, predictable, and easy to make for specific tasks.

Q7. What are their limitations?

Ans: They cannot handle unexpected questions well and do not learn on their own.

Q8. Difference between rule-based and AI-based chatbots?

Ans: Rule-based chatbots follow fixed rules, while AI-based chatbots can use machine learning and NLP to handle more varied messages.

Q9. What happens if no pattern matches?

Ans: The chatbot gives a default reply, such as asking the user to rephrase the question.

Q10. What is NLP?

Ans: NLP stands for Natural Language Processing. It helps computers work with human language.

Q11. Can rule-based chatbots learn?

Ans: Not on their own. A programmer must update their rules or add a learning feature.

Q12. Give two applications of chatbots.

Ans: Two common uses are customer support and answering frequently asked questions.

Assignment 8 - Minimax for Tic-Tac-Toe

Q1. What is the Minimax algorithm, and how does it work in Tic-Tac-Toe?

Ans: Minimax checks possible moves and picks the best one, assuming the opponent also makes the best moves.

Q2. Why is Tic-Tac-Toe considered a suitable game for implementing the Minimax algorithm?

Ans: It is a simple game with a limited number of moves and clear win, loss, or draw results.

Q3. What is the difference between the maximizing player and the minimizing player in Minimax?

Ans: MAX chooses the highest score, while MIN chooses the lowest score.

Q4. How does the evaluation (utility) function determine whether a game state is a win, loss, or draw?

Ans: It gives a positive score for a win, a negative score for a loss, and zero for a draw, from the chosen player's point of view.

Q5. What is the time complexity of the Minimax algorithm, and how can it be improved using Alpha-Beta Pruning?

Ans: Minimax takes O(b^d) time. Alpha-Beta Pruning skips branches that cannot change the result and can reduce the best-case work to about O(b^(d/2)) with good move ordering.

Q6. What are the limitations of the Minimax algorithm when applied to more complex games like chess?

Ans: The number of possible moves grows very quickly, so games like chess need more time and often use depth limits and evaluation rules.